Lab 9 Instructions

Aditya Sarkar

Tulane Dr, Hyattsville, MD | November 4, 2026

1 Helper functions

Every function in this suite relies on strict validation via a shared helper, `_finite`(v, where). Silent arithmetic failures (NaN, $\pm\infty$) and type confusion (such as Python booleans evaluating as integers 0 or 1) represent corrupted pipeline stages.

Whenever any input value violates the following criteria:

1. v must be an instance of `int` or `float`, but explicitly **not** `bool`.
2. v must satisfy $v = v$ (failing if v is NaN).
3. v must satisfy $|v| < \infty$.

The system raises an `AgreementError`. This prevents invalid executions from silently propagating downstream.

2 Performance and Latency Routines

2.1 `latency_report(ms: Iterable[float]) -> dict[str, Any]`

Purpose and Intuition Evaluating an engine's execution duration using only the arithmetic mean hides latency spikes (outliers caused by OS thread preemption, GPU context switches, or thermal throttling). `latency_report` summarizes timing samples by pairing non-parametric percentiles (p_{50}, p_{95}, p_{99}) with the worst-case sample and jitter.

Algorithmic Flow

1. **Validation and Ingestion:** Iterate over `ms`. For each sample x_i, confirm it is finite using `_finite`. Enforce that $x_i \geq 0$. A negative latency indicates non-monotonic clock skew (e.g., taking start and end times from mismatched clocks).
2. **Non-Empty Guard:** If the sequence is empty, raise `AgreementError`. A report cannot be built from zero runs.
3. **Summary Metrics:**
   - Sample count: $n = |\mathbf{x}|$.
   - Mean: $\bar{x} = \frac{1}{n} \sum_{i=0}^{n-1} x_i$.

- Worst-case: $\max(\mathbf{x})$.

4. **Percentile Computation:** For each target quantile $q \in$ LATENCY_PERCENTILES (typically $50, 95, 99$), evaluate the interpolated rank via `_percentile`$(\mathbf{x}, q)$. On a sorted list $\mathbf{s}$:
$$\text{index} = (n-1) \times \frac{q}{100}, \quad \text{lo} = \lfloor \text{index} \rfloor, \quad \text{hi} = \min(\text{lo}+1, n-1)$$
$$p_q = s[\text{lo}] + (s[\text{hi}] - s[\text{lo}]) \times (\text{index} - \text{lo})$$

5. **Jitter Calculation:**
$$\text{jitter} = p_{95} - p_{50}$$
A large jitter relative to the median warns that the median does not reliably describe individual iterations.

Return Schema Returns a dictionary containing keys: `"n"`, `"mean"`, `"worst"`, `"p50"`, `"p95"`, `"p99"`, and `"jitter"`.

2.2 `speedup(base_ms: float, other_ms: float) -> dict[str, Any]`

Purpose and Intuition A multiplicative speedup factor alone can be misleading: achieving a $4\times$ speedup on a 0.02 ms kernel saves negligible wall-clock time, whereas a $1.15\times$ speedup on a 200 ms model saves tens of milliseconds. `speedup` reports both the multiplicative ratio and the absolute wall-clock duration saved.

Algorithmic Flow

1. **Strict Positivity:** Validate both base_ms and other_ms through `_finite`. Both must satisfy $t > 0$. An execution stage that took zero or negative time indicates clock resolution errors or skipped execution.

2. **Ratio Calculation:**
$$\text{factor} = \frac{\text{base_ms}}{\text{other_ms}}$$

3. **Categorical Flag:**
$$\text{faster} = \begin{cases} \text{True} & \text{if factor} > 1.0 \\ \text{False} & \text{otherwise} \end{cases}$$

4. **Absolute Delta:**
$$\text{saved_ms} = \text{base_ms} - \text{other_ms}$$

Return Schema Returns a dictionary containing keys: `"factor"`, `"faster"`, and `"saved_ms"`.

—

3 Numerical Output Deviation

3.1 `max_abs_dev(a: Sequence[float], b: Sequence[float]) -> dict[str, Any]`

Purpose and Intuition Absolute deviation provides an interpretable metric anchored directly in the scale of the target output (such as unnormalized logits or bounding box coordinates). It answers: *What is the single largest divergence between the two output arrays, and precisely where does it occur?*

Mathematical Definition Given two sequences **a** and **b** of length n:

$$\delta_{\text{abs}}(i) = |a_i - b_i|$$

$$\text{max_abs} = \max_{0 \le i < n} \delta_{\text{abs}}(i)$$

Algorithmic Flow

1. **Validation and Ingestion:** Validate every element of **a** and **b** for finiteness. Verify that both sequences are non-empty and have matching lengths: $n = \text{len}(\mathbf{a}) = \text{len}(\mathbf{b})$. Length mismatches point to batching inconsistencies or missing tensor flattening.
2. **Linear Scan with Stable Ties:** Initialize tracking variables at index 0:

   $$\text{best_i} = 0, \quad \text{best} = |a_0 - b_0|$$

   Iterate i from 1 to $n - 1$. Update best and best_i **strictly** when:

   $$|a_i - b_i| > \text{best}$$

   Using a strict inequality ensures deterministic behavior on ties by always preserving the lowest index.

Return Schema Returns a dictionary with:

- `"max_abs"`: The maximum scalar absolute difference ($\max |a_i - b_i|$).
- `"index"`: The flat position i where the peak absolute deviation occurs.
- `"a"`, `"b"`: The specific scalar values a_i and b_i at that index.
- `"n"`: The length of the compared sequences.

3.2 `max_rel_dev(a: Sequence[float], b: Sequence[float], floor: float) -> dict[str, Any]`

Purpose and Intuition Relative error normalizes differences against the magnitude of a reference tensor **b** (e.g., standard FP32 ONNX Runtime). However, when reference values approach zero, standard relative error division explodes, yielding artificially massive deviations. `max_rel_dev` introduces a positive baseline floor $\tau = \text{floor}$ to stabilize the denominator:

$$\text{denom}_i = \max(|b_i|, \tau)$$

Algorithmic Flow

1. **Input Validation:** Verify sequence lengths and numeric finiteness. Validate that the denominator floor satisfies $\tau > 0$. A non-positive floor removes the numerical guard and raises `AgreementError`.
2. **State Tracking:** Maintain counters and minimum values during a single pass:
   - `floored`: Count of indices where $|b_i| < \tau$.
   - `denom_min`: The smallest $|b_i|$ observed across the entire reference array.
   - `best`: The current maximum relative quotient.
   - `best_i`: The lowest index corresponding to `best`.
3. **Pass Computation:** For each index $i \in \{0, \ldots, n-1\}$:

$$d_i = |b_i|, \quad \text{denom_min} \leftarrow \min(\text{denom_min}, d_i)$$

$$\text{if } d_i < \tau \implies \text{floored} \leftarrow \text{floored} + 1$$

$$r_i = \frac{|a_i - b_i|}{\max(d_i, \tau)}$$

   If $r_i >$ best, update best $= r_i$ and best_i $= i$.

Interpretation Notice If `floored` > 0, the quoted relative error over those entries is mathematically an absolute error scaled by a constant factor $\frac{1}{\tau}$. Tracking `floored` and `denom_min` makes this behavior explicit.

—

4 Classification and Decision Stability

4.1 `top1_agreement(rows_a, rows_b) -> dict[str, Any]`

Purpose and Intuition Floating-point drift in logits is only operationally catastrophic if it changes the downstream outcome. `top1_agreement` performs instance-by-instance argmax comparisons to verify that both backends predict the identical top-ranking category.

Algorithmic Flow

1. **Structural Checks:** Ensure the row counts are non-zero and identical: $N = |\text{rows_a}| = |\text{rows_b}|$.
2. **Row Validations:** For each pair of rows $(\mathbf{u}_k, \mathbf{v}_k)$ at sample k:
   - Both rows must contain finite values.
   - Both rows must be non-empty and possess identical class counts ($C = |\mathbf{u}_k| = |\mathbf{v}_k|$).
3. **Argmax with Deterministic Tie-Breaking:** Compute the predicted class indices:

$$i_a = \arg\max_{0 \le c < C} u_k[c], \quad i_b = \arg\max_{0 \le c < C} v_k[c]$$

   Ties must break toward the smallest index c. This matches standard framework behavior (such as PyTorch and NumPy) and ensures deterministic outcomes.

4. **Aggregation:**
   - If $i_a = i_b$: increment agree.
   - If $i_a \neq i_b$ and first_disagreement is unset: record the current row index k.
5. **Rate Calculation:**
$$\text{rate} = \frac{\text{agree}}{N}, \quad \text{disagree} = N - \text{agree}$$

Return Schema Returns a dictionary containing: `"n"`, `"agree"`, `"disagree"`, `"rate"`, and `"first_disagreement"`.

4.2 `decision_margin(rows: Sequence[Sequence[float]], tol: float) -> dict[str, Any]`

Purpose and Intuition If backend A and backend B agree on all current test inputs, could an unobserved input flip decisions? `decision_margin` answers this by measuring the top decision margins of a backend:

$$m_k = \text{top_score}_k - \text{second_highest_score}_k$$

If $m_k >$ tol for all k (where tol is the worst-case absolute deviation max_abs between the backends), then the numerical error is strictly insufficient to flip any classification label in the dataset.

Algorithmic Flow

1. **Validation:** Ensure rows exist and that the deviation tolerance satisfies tol ≥ 0.
2. **Row Evaluation:** For each input vector $\mathbf{r}_k$:
   - Ensure $|\mathbf{r}_k| \geq 2$. A single-class output has no second-best competitor, making decision margin calculation undefined.
   - Sort the entries descending: $s_0 \geq s_1 \geq \cdots \geq s_{C-1}$.
   - Record the margin: $m_k = s_0 - s_1 \geq 0$.
3. **Distribution Statistics:** Over the array of margins $\mathbf{m}$:
   - min $= \min(\mathbf{m})$.
   - argmin $= \arg\min_k m_k$ (choosing the lowest index on ties).
   - median $=$ `_percentile`$(\mathbf{m}, 50)$.
   - mean $= \frac{1}{N}\sum_{k=0}^{N-1} m_k$.
   - below_tol $= \sum_{k=0}^{N-1} \mathbf{1}(m_k <$ tol$)$.
   - safe $=$ (below_tol $== 0$).

Interpretation Notice When `safe` is `True`, the measured difference between the backends cannot flip any classification decisions on this dataset. When `safe` is `False`, `below_tol` identifies how many decisions sit within the margin of error.

—

5 Summary of Function Signatures and Core Metrics

Function	Key Validations	Primary Output Deliverables
`latency_report`	$x_i \geq 0$, finite checks, $n > 0$	n, mean, worst, p_{50}, p_{95}, p_{99}, jitter
`speedup`	base, other > 0	factor ($> 1 \implies$ faster), saved wall-clock ms
`max_abs_dev`	Equal length, finite entries	Peak absolute error, lowest tie index, values
`max_rel_dev`	Floor $\tau > 0$, equal length	Clamped relative error, floor hits, minimum reference
`top1_agreement`	Row count & width match, finite entries	Agreement rate, count of mismatches, first divergence row
`decision_margin`	Width ≥ 2, tol ≥ 0	Smallest margin, mean/median, safety certificate flag

Table 1: Overview of verification and profiling functions.